```
!pip install pandas nltk scikit-learn -q
```

```
import pandas as pd
import nltk
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
True
```

```
data = {
    "text": [
        "I love this product",
        "This is the worst experience",
        "Amazing service and great quality",
        "I hate this item",
        "Very happy with the purchase",
        "Not good, very disappointing",
        "Excellent work",
        "Terrible and useless product",
        "I am satisfied",
        "Bad quality and broken"
    ],
    "sentiment": [
        "positive",
        "negative",
        "positive",
        "negative",
        "positive",
        "negative",
        "positive",
        "negative",
        "positive",
        "negative"
    ]
}

df = pd.DataFrame(data)
df
```

	text	sentiment
0	I love this product	positive
1	This is the worst experience	negative
2	Amazing service and great quality	positive
3	I hate this item	negative
4	Very happy with the purchase	positive
5	Not good, very disappointing	negative
6	Excellent work	positive
7	Terrible and useless product	negative
8	I am satisfied	positive
9	Bad quality and broken	negative

Next steps: Generate code with df | New interactive sheet

```
X = df['text']
y = df['sentiment']

vectorizer = CountVectorizer(stop_words='english')
X_vectorized = vectorizer.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_vectorized, y, test_size=0.2, random_state=42
)
```

```
model = MultinomialNB()
model.fit(X_train, y_train)
```

```
▾ MultinomialNB
MultinomialNB()
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nReport:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.5

Report:
              precision    recall  f1-score   support

    negative       0.50      1.00      0.67         1
    positive       0.00      0.00      0.00         1

    accuracy                           0.50         2
   macro avg       0.25      0.50      0.33         2
weighted avg       0.25      0.50      0.33         2

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.p
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
def predict_sentiment(text):
    text_vec = vectorizer.transform([text])
    prediction = model.predict(text_vec)
    return prediction[0]

print(predict_sentiment("I really love this!"))
print(predict_sentiment("This is very bad and useless"))
```

```
positive
negative
```